

Competitive Programming

Section 12: Stacks, Queues, And Deques

Competitive Programming Course

2026-07-20

Table of contents

1	Why these structures matter	3
2	The three containers	4
3	Monotonic stack	5
4	Why the pop is safe	6
5	Monotonic deque for sliding windows	7
6	Deque over prefix sums	8
6.1	Turn subarrays into pairs of prefix indices	8
6.2	What the deque stores	9
7	Worked prefix example	10
8	Common recognition signals	11
9	C++ implementation notes	12
10	Python implementation notes	13
11	Proof habits	14
12	Common wrong approaches	15
12.1	Quadratic nearest-element scan	15
12.2	Forgetting expiration	15
12.3	Keeping values instead of indices	15
12.4	Using two pointers with negative numbers	15
13	Local judge workflow	16
14	Checklist before submitting	17
15	Exercises	18
15.1	A. Next Greater	18
15.2	B. Sliding Window Maximum	18
15.3	C. Shortest Subarray At Least K	18

1 Why these structures matter

Stacks, queues, and deques are not only basic containers. In contests they are often used to keep exactly the candidates that can still matter.

The pattern is:

```
When a new element arrives, remove old candidates made useless by it.
```

This gives linear-time algorithms for problems that look quadratic:

- nearest greater or smaller element;
- sliding window maximum or minimum;
- removing adjacent pairs;
- shortest subarray with a target sum even when values are negative.

2 The three containers

Container	Access pattern	Typical contest use
Stack	last in, first out	nested structure, nearest element
Queue	first in, first out	simulation, BFS later
Deque	push/pop both ends	sliding windows, monotonic candidates

In C++:

```
vector<int> st;      // often best as a stack
queue<int> q;
deque<int> dq;
```

In Python:

```
stack = []
from collections import deque
dq = deque()
```

3 Monotonic stack

A monotonic stack stores candidates in sorted order by value.

For next greater element to the right, scan from right to left and keep a stack of values that are strictly greater candidates.

When processing $a[i]$, any stack top $\leq a[i]$ is useless:

$a[i]$ is closer to the left and at least as large, so that old value can never be the first greater value for any earlier element.

Example:

```
a = 2 1 3 2 5
```

Scan from right:

```
i=4, value=5: stack empty -> ans=-1, push 5
i=3, value=2: top 5 > 2 -> ans=5, push 2
i=2, value=3: pop 2, top 5 > 3 -> ans=5, push 3
i=1, value=1: top 3 > 1 -> ans=3, push 1
i=0, value=2: pop 1, top 3 > 2 -> ans=3, push 2
```

Answer:

```
3 3 5 5 -1
```

4 Why the pop is safe

Suppose we scan from right to left and see current value x .

If the stack top is $y \leq x$, then y is to the right of x , but x is closer to every future position we will process. Since x is at least as large as y , y can never be the first strictly greater value for a future position.

So y can be removed permanently.

That is the core amortized idea:

`each element is pushed once and popped once`

5 Monotonic deque for sliding windows

For window maximums, the front of the deque should always hold the index of the maximum value in the current window.

Maintain two invariants:

1. indices in the deque are inside the current window;
2. values in the deque are decreasing from front to back.

When a new index i arrives:

1. Remove front indices that are too old.
2. Remove back indices whose values are $\leq a[i]$.
3. Push i .
4. Once the first full window exists, the answer is $a[dq.front()]$.

Example:

```
a = 1 3 -1 -3 5 3 6 7
k = 3
```

Window maximums:

```
[1, 3, -1] -> 3
[3, -1, -3] -> 3
[-1, -3, 5] -> 5
[-3, 5, 3] -> 5
[5, 3, 6] -> 6
[3, 6, 7] -> 7
```

6 Deque over prefix sums

This is for a harder-looking task:

```
Find the shortest non-empty subarray whose sum is at least K.
```

If every number were nonnegative, ordinary two pointers would work. With a negative number, moving the right end rightward can *decrease* the sum, so that usual reasoning breaks.

6.1 Turn subarrays into pairs of prefix indices

Build a prefix array with one extra entry. For $0 \leq i < n$:

```
pref[0] = 0
pref[i + 1] = pref[i] + a[i]
```

For example, if the array begins $a[0]$, $a[1]$, $a[2]$, ..., then:

```
pref = [0, a[0], a[0] + a[1], a[0] + a[1] + a[2], ...]
```

So $\text{pref}[i]$ equals $a[0] + \dots + a[i - 1]$: the sum of the first i elements. For an ordinary inclusive subarray from **start** through **end**:

```
sum(a[start ... end]) = pref[end + 1] - pref[start]
length                  = end - start + 1
```

For the deque algorithm, it is more convenient to process every prefix index **right** from 0 through n . Pairing an earlier index **start** with **right** means the subarray $a[\text{start} \dots \text{right} - 1]$:

```
sum:    pref[right] - pref[start]
length: right - start
```

So, while processing $\text{pref}[\text{right}]$, we seek earlier indices **start** satisfying:

```
pref[right] - pref[start] >= K
```

Among valid **start** values, a later one gives a shorter subarray. We need candidates that are both useful for the sum and as far right as possible.

6.2 What the deque stores

The deque stores **prefix indices**, not array values. Their prefix sums are strictly increasing from front to back:

```
dq = [j0, j1, j2, ...]
pref[j0] < pref[j1] < pref[j2] < ...
```

At the current prefix index **right**, do these operations in this order:

1. While $\text{pref}[\text{right}] - \text{pref}[\text{dq.front}()] \geq K$, record the length $\text{right} - \text{dq.front}()$ and pop the front.
2. While $\text{pref}[\text{dq.back}()] \geq \text{pref}[\text{right}]$, pop the back.
3. Append **right**.

The two kinds of pop have different reasons.

Pop from the front after finding a valid subarray. The front is the oldest candidate. It has just formed a valid subarray ending at index **right** - 1. Any later **right** would make that candidate's length larger, never smaller. It has no future use, so discard it and try the next candidate; that next candidate is later and therefore gives a shorter length if it is also valid.

Pop from the back before appending right. Suppose an old back index **j** has $\text{pref}[\text{j}] \geq \text{pref}[\text{right}]$. Index **right** is later and has an equal or smaller prefix sum. For every future prefix index **future**:

```
pref[future] - pref[right] >= pref[future] - pref[j]
future - right < future - j
```

Thus **right** gives at least as much sum and a strictly shorter subarray. Index **j** is dominated forever and can be removed.

7 Worked prefix example

Find the shortest subarray with sum at least $K = 5$:

```
index:  0   1   2   3   4   5
a:      2  -1   2  -1   2   4
```

Prefix sums:

```
prefix index:  0   1   2   3   4   5   6
pref:         0   2   1   3   2   4   8
```

Start with an empty deque. The following trace writes the prefix indices in the deque, followed by their prefix values in parentheses.

```
i = 0, pref = 0: append 0
dq = [0]                                     (0)

i = 1, pref = 2: no valid front; append 1
dq = [0, 1]                                 (0, 2)

i = 2, pref = 1: no valid front; pop 1 because 2 >= 1; append 2
dq = [0, 2]                                 (0, 1)

i = 3, pref = 3: no valid front; append 3
dq = [0, 2, 3]                             (0, 1, 3)

i = 4, pref = 2: no valid front; pop 3 because 3 >= 2; append 4
dq = [0, 2, 4]                             (0, 1, 2)

i = 5, pref = 4: no valid front; append 5
dq = [0, 2, 4, 5]                         (0, 1, 2, 4)
```

Now process $i = 6$, where $\text{pref}[6] = 8$. The deque front is truly index 0, not index 4. Repeatedly use the front while it gives a sum of at least 5:

```
front 0: 8 - pref[0] = 8 -> length 6 - 0 = 6; pop 0
front 2: 8 - pref[2] = 7 -> length 6 - 2 = 4; pop 2
front 4: 8 - pref[4] = 6 -> length 6 - 4 = 2; pop 4
front 5: 8 - pref[5] = 4 -> stop: this sum is too small
```

The best length is 2. Here $\text{start} = 4$ and $i = 6$, so the actual subarray is $a[4:6]$, namely $2 + 4 = 6$.

The important pattern is: a prefix index enters the deque once, then can leave once from the front or once from the back. Therefore the deque work is linear; sorting is not needed.

8 Common recognition signals

Use a monotonic stack when the statement asks for:

- next greater;
- next smaller;
- nearest taller building;
- stock span;
- remove dominated candidates to the left or right.

Use a monotonic deque when the statement asks for:

- maximum/minimum over every fixed-size window;
- shortest/longest interval with a prefix-sum condition;
- DP transition over a sliding range.

Use a plain queue when the statement is a literal FIFO process or, later, BFS.

9 C++ implementation notes

For stacks, `vector<T>` is often faster and more flexible than `stack<T>`:

```
vector<long long> st;  
st.push_back(x);  
st.pop_back();  
st.back();
```

For deques:

```
deque<int> dq;  
dq.push_back(i);  
dq.pop_front();  
dq.pop_back();
```

Use indices, not values, when you must know whether an element has left a window.

10 Python implementation notes

Use a list as a stack:

```
stack.append(x)
stack.pop()
stack[-1]
```

Use `collections.deque` for efficient front pops:

```
from collections import deque
dq = deque()
dq.popleft()
```

Avoid `list.pop(0)`; it is linear time.

11 Proof habits

For monotonic structures, prove two things:

1. **Dominance.** When you pop a candidate, explain why it can never be useful again.
2. **Amortization.** Each element is inserted once and removed at most once, so total time is linear after input reading.

The code is short, but the proof is the reason it is trusted.

12 Common wrong approaches

12.1 Quadratic nearest-element scan

Checking all positions to the right for each index is too slow for $n=200000$.

12.2 Forgetting expiration

For sliding windows, a large value can stay in the deque after it leaves the window unless you remove old indices from the front.

12.3 Keeping values instead of indices

For window problems, values alone do not tell you whether the candidate is still inside the window.

12.4 Using two pointers with negative numbers

If numbers can be negative, window sums are not monotone under expansion. Use prefix sums and a deque instead.

13 Local judge workflow

Run Section 12:

```
python3 sections/12_stacks_queues_deques/check.py
```

Run the reference solutions:

```
CP_TARGET=solution python3 sections/12_stacks_queues_deques/check.py
```

The random tests compare against brute force on small arrays. That is useful for this topic because off-by-one expiration and equality choices are common.

14 Checklist before submitting

- Decide whether equality should be popped: $<$, $\leq$, $>$, or $\geq$.
- Store indices when window membership matters.
- Remove expired indices before reading the front.
- Use prefix sums for subarray sums.
- Use 64-bit integers for prefix sums in C++.
- Prove every popped element is dominated.
- Test duplicates, all increasing, all decreasing, and negative values.

15 Exercises

15.1 A. Next Greater

Practice a right-to-left monotonic stack.

15.2 B. Sliding Window Maximum

Practice a decreasing deque of indices.

15.3 C. Shortest Subarray At Least K

Practice a monotonic deque over prefix sums, including negative values.